160226

February 23, 2026

0.0.1 Imports

[2]:
```
## Nessacery imports for entire lab done here for easy management
from scipy.optimize import minimize
import numpy as np
import matplotlib.pyplot as plt
import hyperspy.api as hs
from scipy import optimize
```

0.0.2 Functions

[3]:
```
## Defining perodic function with a sine and cosine term, variables allow␣
↪manipulation of all characteristics,
## while the presence of a sin and cos allow variation of amplitude etc in a␣
↪period
def perodic(x,a,b,c,d,e,f,g):
    return (a*np.sin(b*np.deg2rad(x) + c) + (d*np.cos(e*np.deg2rad(x) + f)) + g)


## Definig perodic plot function to fit the perodic function to one slice of␣
↪data
def periodic_fit_plot(x_variable, y_variable,initial, plot = False):
## Using optimize.curve_fit to fit a perodic funtion to our x and y data,␣
↪intial values for parameters taken from input
    params_sin, params_sin_covariance = optimize.
↪curve_fit(perodic,x_variable,y_variable,p0=initial,maxfev=30000)

## Finding estimated uncertainties for each parameter
    errs = np.sqrt(np.diag(params_sin_covariance))

## creating a dummy variable for plotting purposes using linspace, parameters␣
↪of linspace ensure it varies from lowest to highest x value regarless of␣
↪order in array
    dummy = np.linspace(np.min(x_variable),np.max(x_variable),360)

## Calculating residuals
    difference = perodic(x_variable,*params_sin) - y_variable
```

```
## Calculating r squared for goodness of fit

## Squared sum of residuals
    ss_res = np.sum(difference**2)

## Squared sum of data - mean
    ss_tot = np.sum((y_variable- np.mean(y_variable))**2)

## Calculate r_squared per equation
    r_squared = 1 - (ss_res/ss_tot)

    ## Rounding frequencies, ensuring single values at 0/360
    params_sin[1] = np.round(params_sin[1], decimals=0)
    params_sin[4] = np.round(params_sin[4], decimals=0)
    if plot == True:
## Setting up subplots
        plt.subplots(2, 1, gridspec_kw={'height_ratios':␣
 ↪[2,1]},sharex=True,figsize=[10,6]) # height ratios make sizes right, sharex␣
 ↪makes plots share x axis
## Subplot 2 shows residuals
        plt.subplot(212)
        plt.plot(x_variable,difference,'or')
        plt.xlabel("Current (A)")
        plt.ylabel("Residuals")

        plt.grid()
        plt.tight_layout()
## Subplot 1 with fit and raw data
        plt.subplot(211)
        plt.scatter(x_variable, y_variable,label="Raw Data")
        plt.plot(dummy,perodic(dummy,*params_sin),color="r",label="Fitted␣
 ↪Function") ## using curvefit parameters and linear function to plot function
        plt.
 ↪fill_between(dummy,perodic(dummy,*(params_sin+errs)),perodic(dummy,*(params_sin-errs)),alph
 ↪2) ## highlights linear fit within 1 SD of both parameters, too small to see␣
 ↪for this data
        plt.plot([], [], ' ', label="R^2 = {0:.4f}".format(r_squared)) ##␣
 ↪displaying r squared value
        plt.grid()
        plt.legend()

    return(params_sin,errs)
```

[4]:
```
## Defining a function to iterate previous function over the whole data set and␣
 ↪plot the parameters for each slice

def periodic_fit_plot_whole(dataset,initial, plot = False):
```

```
    import matplotlib.pyplot as plt
    import matplotlib.gridspec as gridspec

## Opening empty arrays to fill with parameters and associated errors
    params_array = []
    errors_array = []

    degrees = np.linspace(0,360,360)
## Beginning for loop to iterate over every slice of data
    for x in range(len(dataset[:,0,0])): # This begins loop for length of one␣
 ↪dimension
        for y in range(len(dataset[0,:,0])): # For every value in first␣
 ↪dimension we iterate over all slices in the 2nd
            params = periodic_fit_plot(degrees, dataset[x,y,:],initial, plot =␣
 ↪False) # Runs previous function on each slice
            params_array.append(params[0])
            errors_array.append(params[1])
    return params_array, errors_array
```

[5]:
```
def decomp_data(path, components, algo):
    raw_data = hs.load(path)
    raw_data_end = np.load(path)
    data_hs_func = hs.signals.Signal1D(raw_data.data)
    data_hs_func.decomposition(algorithm=algo)
    sc = data_hs_func.get_decomposition_model(components)
    decomposed_data = sc.data

    return decomposed_data, raw_data_end
```

[6]:
```
def decomp_and_fit(path, components, algo):
    decomp = decomp_data(path,components,algo)

    decomp_params =␣
 ↪periodic_fit_plot(x,decomp[0][0][20],[20,1,1,20,1,1,5],plot=False)
    params = periodic_fit_plot(x,decomp[1][0][20],[20,1,1,20,1,1,5],plot=False)

    return decomp[0], decomp[1], decomp_params, params
```

[7]:
```
def find_elbow(varience):
    """
    Find elbow point in scree plot data.

    Parameters:
        varience : array - Varience explained values

    Returns:
        elbow_x : component at elbow
```

```
        elbow_index : index of elbow
        relevant_components : components to keep
    """

    x = np.linspace(1,len(varience),len(varience))
    y = np.array(varience)

    # Normalize data (important for generic datasets)
    x_norm = (x - x.min()) / (x.max() - x.min())
    y_norm = (y - y.min()) / (y.max() - y.min())

    # Line between first and last points
    line_vec = np.array([x_norm[-1] - x_norm[0],
                         y_norm[-1] - y_norm[0]])
    line_vec = line_vec / np.linalg.norm(line_vec)

    # Compute distance from each point to the line
    distances = []
    for i in range(len(x_norm)):
        point_vec = np.array([x_norm[i] - x_norm[0],
                              y_norm[i] - y_norm[0]])

        proj_len = np.dot(point_vec, line_vec)
        proj_vec = proj_len * line_vec

        dist_vec = point_vec - proj_vec
        distances.append(np.linalg.norm(dist_vec))

    elbow_index = np.argmax(distances)
    elbow_x = x[elbow_index]

    relevant_components = x[:elbow_index + 1]

    return elbow_x, elbow_index, relevant_components
```

[8]:
```
## Adjusted R square
import numpy as np

def adjusted_r2(y_true, y_pred, n_params):
    """
    Compute adjusted R-squared for a model.

    Parameters
    ----------
    y_true : array - True Data
    y_pred : array - Fit Predictions
    n_params : int - Number of parameters in fit
```

```
    Returns
    -------
    r2_adj : float - Adjusted R-squared.
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    n = len(y_true)

    # Residual sum of squares
    ss_res = np.sum((y_true - y_pred) ** 2)

    # Total sum of squares
    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)

    # Standard R^2
    r2 = 1 - ss_res / ss_tot

    # Adjusted R^2
    r2_adj = 1 - (1 - r2) * (n - 1) / (n - n_params - 1)

    return r2_adj
```

0.0.3 Data Processing

[ ]:
```
## This notebook covers the first data testing of the newest LSAT dataset.␣
↪First uses of find_elbow and adj r square functions
```

[9]:
```
LSAT_data = np.load("binned_diff_20210421_162329_fast_2026_PT1D.npy")
```

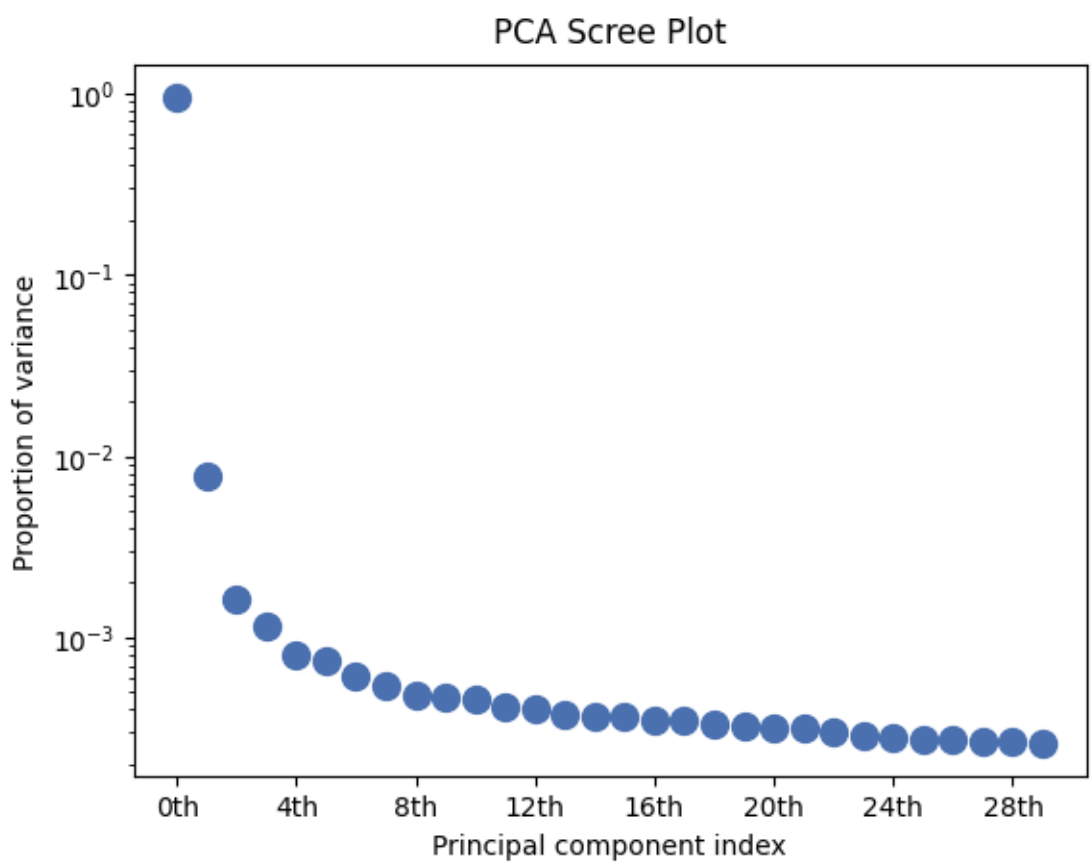
[10]:
```
data_hs1 = hs.signals.Signal1D(LSAT_data.data)
data_hs1.decomposition()
```

```
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
```

```
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
WARNING | Hyperspy | Numba is not installed, falling back to non-
accelerated implementation. (hyperspy.decorators:256)
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
```

[11]:
```
## Plotting scree plot
data_hs1.plot_explained_variance_ratio()
```

[11]:
```
<Axes: title={'center': '\nPCA Scree Plot'}, xlabel='Principal component index',
ylabel='Proportion of variance'>
```



[13]:
```
scree_plot = data_hs1.get_explained_variance_ratio().data
```

[17]:
```
relavent = find_elbow(scree_plot)[2]
print(relavent)
```

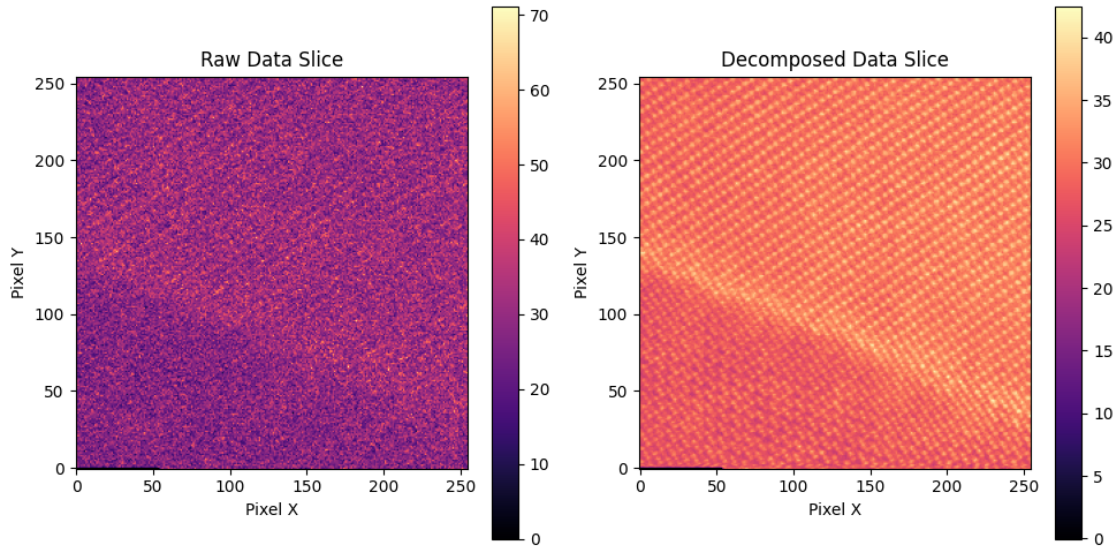
```
[1. 2. 3.]
```

[19]:
```
### Comparing visual scree plot and result from function they agree that the␣
↪first 3 components of the decomposition hold meaning. This gives confidence␣
↪in the function.
```

[18]:
```
sd = decomp_data("binned_diff_20210421_162329_fast_2026_PT1D.
↪npy",len(relavent),"SVD")
```

```
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
```

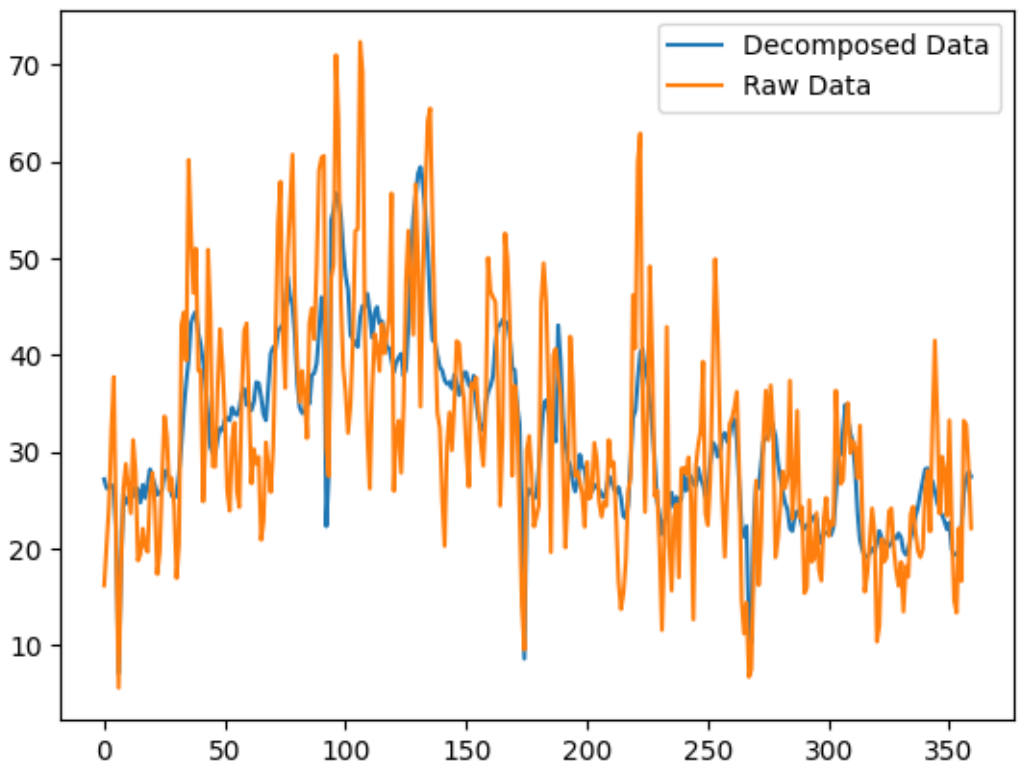
[23]:
```
## Using imshow to visualise slice of both raw and decomposed datasets

slice_index = 10  # azimuthal slice to visualize

fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Raw
im0 = axes[0].imshow(sd[1][:, :, slice_index], cmap='magma', origin='lower')
axes[0].set_title('Raw Data Slice')
plt.colorbar(im0, ax=axes[0])

# Decomp
im1 = axes[1].imshow(sd[0][:, :, slice_index], cmap='magma', origin='lower')
axes[1].set_title('Decomposed Data Slice')
plt.colorbar(im1, ax=axes[1])

# Common labels
for ax in axes:
    ax.set_xlabel('Pixel X')
    ax.set_ylabel('Pixel Y')

plt.tight_layout()
plt.show()
```



Based on the visual appearance of the raw and decomposed data, we see a large improvement in the appearance of the periodic pattern when going from raw to decomposed.

[155]:
```
## Now showing a chosen pixel to compare the data through the 360 range as a 2d␣
↪data set.
plt.plot(sd[0][50][50],label="Decomposed Data")
plt.plot(sd[1][50][50],label="Raw Data")
plt.legend()
```

[155]: <matplotlib.legend.Legend at 0x7f08755e26d0>

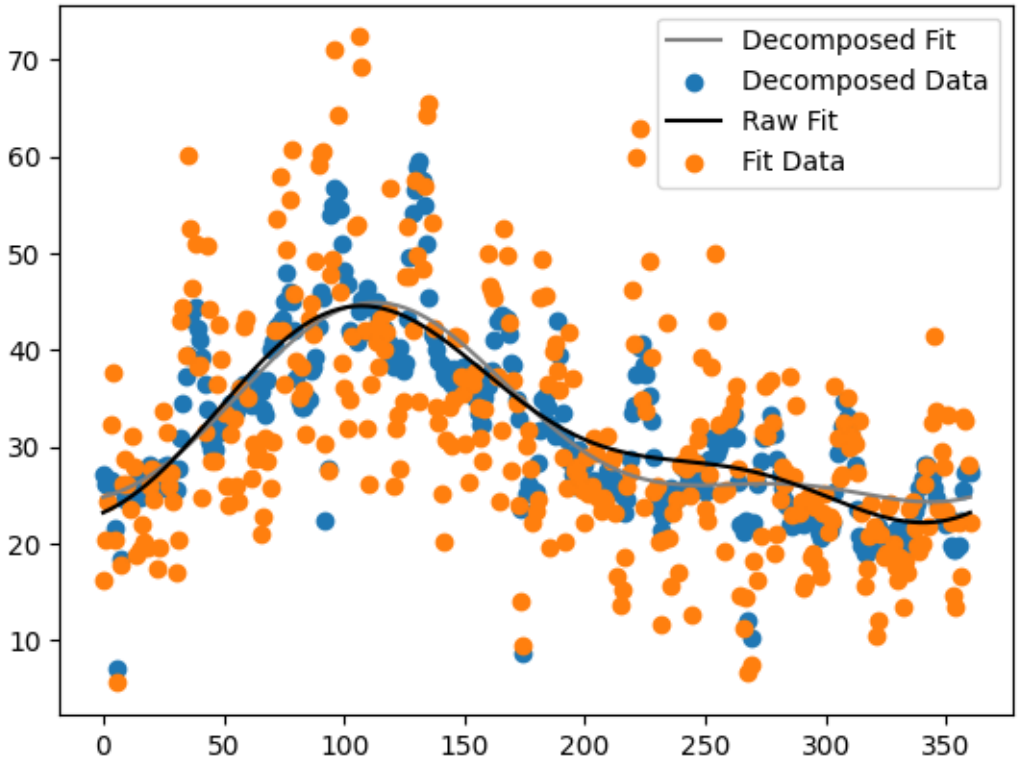


The data from this pixel retains its shape and pattern when decomposed, but with less noise, this is what we are aiming for with decomposition.

[24]:
```
## Fitting periodic function to compare raw and decomped data

x = np.linspace(0,360,360)
slice_num = [50,50]
decomped = sd[0][slice_num[0]][slice_num[1]]
raw = sd[1][slice_num[0]][slice_num[1]]

de_params = periodic_fit_plot(x,decomped,[60,1,1,1,2,1,1])
raw_params = periodic_fit_plot(x,raw,[70,1,1,1,2,1,1])
```

[25]:
```
## Plotting both sets of data and their corresponding fits

plt.plot(x,perodic(x,*de_params[0]),color="grey",label="Decomposed Fit")
plt.scatter(x,decomped,label="Decomposed Data")

plt.plot(x,perodic(x,*raw_params[0]),color="black",label="Raw Fit")
plt.scatter(x,raw,label="Fit Data")
plt.legend()
```

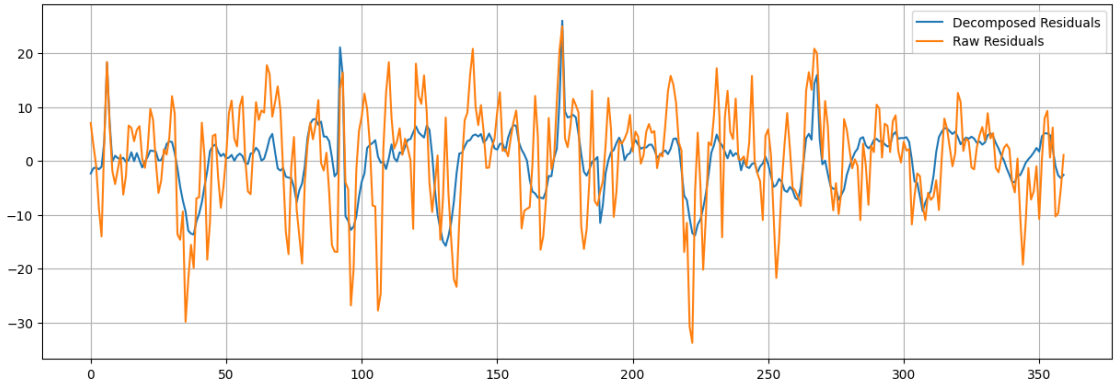
[25]: <matplotlib.legend.Legend at 0x7f4199f9e6d0>



Not a large difference in fits between the datasets. Potentially what we expect as they are measuring the same underlying function.

[171]:
```
### Residual Calculations

difference_decomp = perodic(x,*de_params[0]) - decomped
difference_raw = perodic(x,*raw_params[0]) - raw

plt.figure(figsize=[15,5])
plt.plot(difference_decomp,label="Decomposed Residuals")
plt.plot(difference_raw,label="Raw Residuals")
plt.grid()
plt.legend()
```

[171]: <matplotlib.legend.Legend at 0x7f0874d23790>



No clear change in the residual patterns

[168]:
```
## R Squared Values

print(adjusted_r2(decomped,perodic(x,*de_params[0]),7))
print(adjusted_r2(raw,perodic(x,*raw_params[0]),7))
```

```
0.6273752043105837
0.3405875088375899
```

Significantly better adjusted r squared value for the decomped fit, this gives more confidence in the new fit.